FastBite

(notre application de livraison de repas, type Deliveroo)

TP5 — Exceptions, fichier texte, sérialisation / désérialisation

[Prénom NOM 1] · [Prénom NOM 2] · [Prénom NOM 3]

Programmation orientée objet en Java — ECE Paris, Ing2

Enseignant : J.-P. Segado · 02/10/2026

Sommaire

1 **Q1 — Exceptions**
repérer, lever, propager, attraper + UML mis à jour (3 slides)

2 **Q2 — Fichier texte**
écrire, lire et afficher le menu d'un restaurant (2 slides)

3 **Q3 — Sérialisation**
enregistrer et relire des restaurants dans un .ser (2 slides)

4 **Bilan, sources et IA**
ce qui marche, ce qui a changé, outils utilisés

Point de départ : notre TP4 complet (Evaluable, Utilisateur abstraite, Panier extends Commande implements Comparable). Le TP5 le modifie le moins possible.

Q1 — Exceptions : repérer, lever, propager

Exception (extends FastBiteException)	Levée par	Pourquoi
NoteInvalideException	ajouterNote (Livreur, Plat, Restaurant) ; propagée par Client.noterRider	note hors 0–5, NaN
AucuneNoteException	getNoteMoyenne	aucune note (le TP4 renvoyait 0.0)
PaiementImpossibleException	Client.payerCommande	commande vide ou déjà payée
CapaciteDepasseeException	Client.passerCommande, Restaurant.gererMenu	tableaux de 20 pleins
LivreurIndisponibleException	Livreur.accepterLivraison	livreur déjà occupé
PlatIndisponibleException	Commande.ajouterPlat (et Panier)	plat absent du menu
CommandeVideException	Commande.valider	aucun plat
EtatLivraisonException	Livraison.demarrer / terminer	étape dans le mauvais ordre
FormatFichierException	Restaurant.chargerMenu (Q2)	ligne du fichier mal formée

• Non vérifiée, conservée du TP4 : IllegalArgumentException (plat null) — c'est un bug du code appelant, pas une situation à « gérer ».

• Rien n'est attrapé dans les méthodes métier : elles lèvent ou propagent (throws), le Main attrape.



• FastBiteException.java

```java
public class FastBiteException extends Exception {
    private static final long serialVersionUID = 1L;


    public FastBiteException() {
        super();
    }

    public FastBiteException(String message) {
        super(message);
    }

    // Constructeur avec cause : garde la trace de l'exception d'origine (ex : NumberFormatException)
    public FastBiteException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

• NoteInvalideException.java

```java
public class NoteInvalideException extends FastBiteException {
    private static final long serialVersionUID = 1L;

    private final double note;

    public NoteInvalideException(double note) {
        super("Note invalide : " + note
            + " (attendu entre " + Evaluable.NOTE_MIN + " et " + Evaluable.NOTE_MAX + ")");
        this.note = note;
    }

    public double getNote() { return note; }
}
```

• Evaluable.java

```java
public interface Evaluable {
    // Bornes des notes (constantes : public static final implicites dans une interface)
    double NOTE_MIN = 0.0;
    double NOTE_MAX = 5.0;

    // TP4 : IllegalArgumentException (non verifiee)  ->  TP5 : NoteInvalideException (verifiee)
    void ajouterNote(double note) throws NoteInvalideException;

    // TP4 : retournait 0 sans note (ambigu avec une vraie note 0)  ->  TP5 : AucuneNoteException
    double getNoteMoyenne() throws AucuneNoteException;
}
```

Capture 1 — la hiérarchie et l'interface Evaluable avec throws

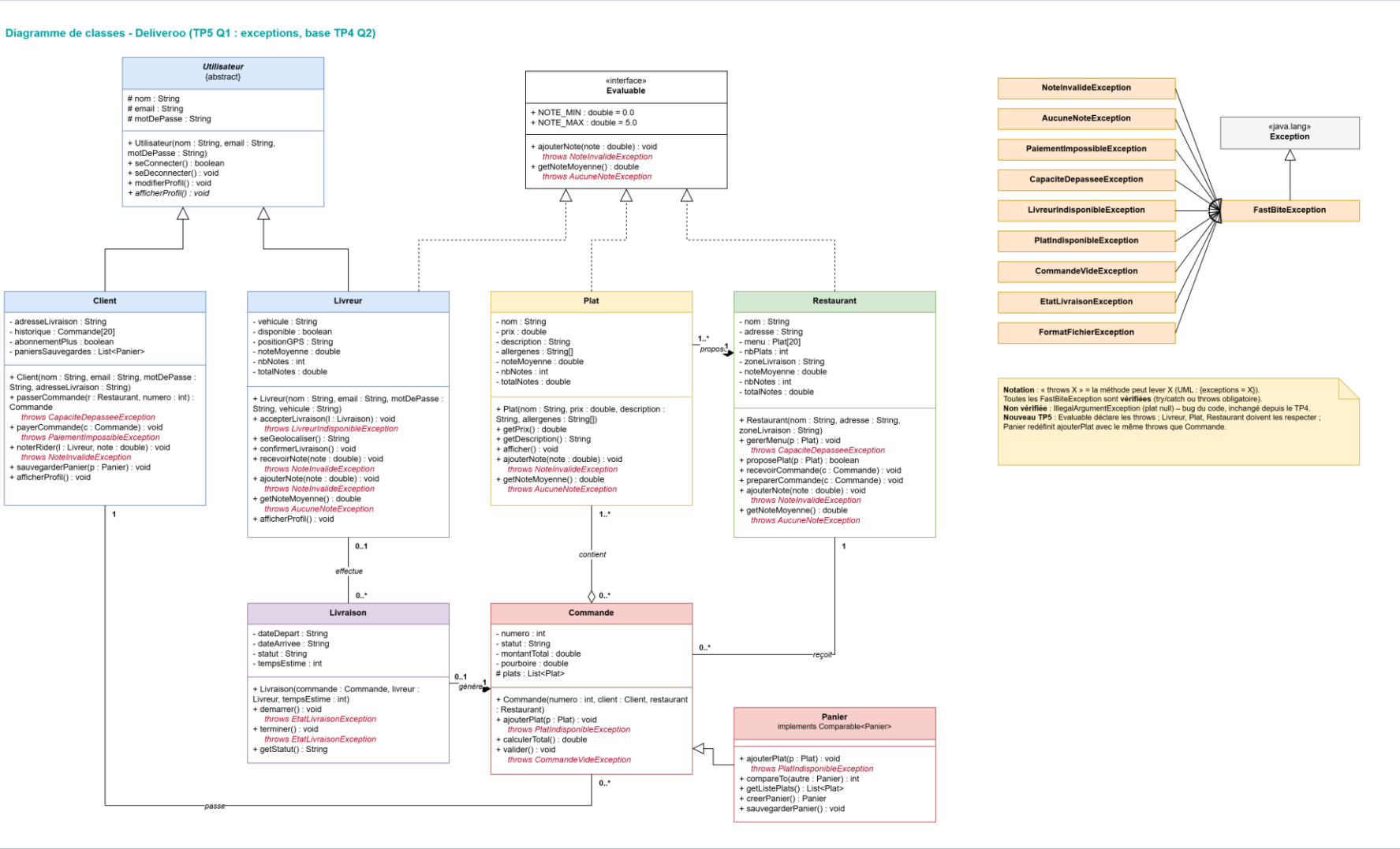
Pourquoi vérifiées (checked) ?

• Erreurs prévisibles et récupérables : le compilateur oblige à les traiter.

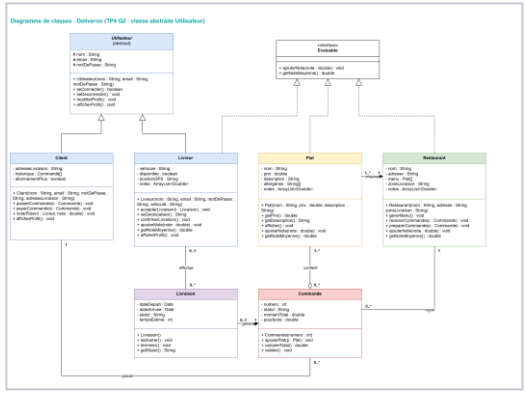
• Une classe mère FastBiteException : un seul catch suffit pour toutes les erreurs métier.

• Changement TP4 : IllegalArgumentException → NoteInvalideException ; moyenne 0.0 → AucuneNoteException.

Q1 — UML mis à jour (base : notre diagramme TP4)



Capture 2 — notre diagramme TP4 complété : « throws » dans les signatures + hiérarchie des exceptions



Avant : diagramme TP4 (base)

Ce qui change

- « throws X » rouge : exceptions que la méthode peut lever.
- Colonne orange : FastBiteException + 9 sous-classes.
- Panier (TP4 Q3) ajouté ; il redéclare le throws de Commande.
- Signatures alignées sur le code réel.

Q1 — Lever, propager, attraper : code et résultat

• Livreur.java

```java
public void accepterLivraison(Livraison l) throws LivreurIndisponibleException {
    if (!disponible) {
        throw new LivreurIndisponibleException(nom);
    }
    disponible = false;
    l.setStatut("EN_COURS");
    System.out.println(nom + " (" + vehicule + ") accepte la livraison #"
            + l.getCommande().getNumero());
}
```

• Commande.java

```java
public void ajouterPlat(Plat p) throws PlatIndisponibleException {
    if (p == null) throw new IllegalArgumentException("Plat manquant.");
    if (restaurant != null && !restaurant.proposePlat(p)) {
        throw new PlatIndisponibleException(p.getNom(), restaurant.getNom());
    }
    plats.add(p);
    System.out.println("  + " + p + " ajoute a la commande #" + numero);
}
```

• Commande.java

```java
public void valider() throws CommandeVideException {
    if (plats.isEmpty()) {
        throw new CommandeVideException(numero);
    }
    calculerTotal();
    this.statut = "VALIDEE";
    System.out.println("Commande #" + numero + " validee (total : " + montantTotal + " EUR)");
}
```

• Livreur.java

```java
public void ajouterNote(double note) throws NoteInvalideException {
    if (Double.isNaN(note) || Double.isInfinite(note) || note < NOTE_MIN || note > NOTE_MAX) {
        throw new NoteInvalideException(note);
    }
    totalNotes += note;
    nbNotes++;
    noteMoyenne = totalNotes / nbNotes;
}
```

Capture 3 — throw / throws dans Livreur et Commande

Windows PowerShell

```
PS ...\semaine 5\tp5_fastbite> java -cp out Main q1
Pizza Margherita ajoute au menu de Chez Mario
Pasta Carbonara ajoute au menu de Chez Mario

>>> B1. Note invalide (NoteInvalideException)
Attrape : Note invalide : 7.5 (attendu entre 0.0 et 5.0)

>>> B2. Moyenne sans aucune note (AucuneNoteException)
Attrape : Aucune note enregistree pour le livreur Bob : moyenne impossible

>>> B3. Plat absent du menu (PlatIndisponibleException)
Alice passe la commande #2001 chez Chez Mario
Attrape : Le plat "Sushi saumon" n'est pas au menu de Chez Mario

>>> B4. Commande vide : validation et paiement impossibles
Attrape : La commande #2001 est vide : validation impossible
Attrape : Paiement impossible pour la commande #2001 : la commande est vide

>>> B5. Livreur deja occupe (LivreurIndisponibleException) + finally
  + Pizza Margherita (12.5 EUR) ajoute a la commande #2001
Livraison generee pour la commande #2001
Bob (scooter) accepte la livraison #2001
Alice passe la commande #2002 chez Chez Mario
  + Pizza Margherita (12.5 EUR) ajoute a la commande #2002
Livraison generee pour la commande #2002
Attrape : Le livreur Bob n'est pas disponible
finally : execute dans tous les cas (exception ou non)

>>> B6. Livraison dans un etat incoherent (EtatLivraisonException)
Attrape : Livraison #2002 : impossible de terminer la livraison (statut actuel : EN_ATTENTE)

>>> B7. Capacite depassee : 21e plat dans un menu de 20 (CapaciteDepasseeException)
Attrape : Capacite depassee : le menu de Resto plein ne peut pas contenir plus de 20 elements
Plats effectivement ajoutes : 20

>>> B8. Exceptions NON verifiees (IllegalArgumentException) : bugs du code appelant
Attrape : le plat ne peut pas etre null
Attrape : Plat manquant.

PS ...\semaine 5\tp5_fastbite>
```

Capture 4 — chaque exception est levée puis attrapée (B1 à B8) ; finally exécuté

Q2 — Écrire et lire un fichier texte

Format du fichier (menu_chezmario.txt)

- 1 ligne = 1 plat, 4 champs séparés par « ; »
- **nom;prix;description;allergène1,allergène2**
- prix avec point décimal ; allergènes séparés par « , » (vide = aucun)

• Restaurant.java

```
151    public void sauvegarderMenu(String fichier) throws IOException {
152        try (BufferedWriter out = new BufferedWriter(new FileWriter(fichier))) {
153            for (int i = 0; i < nbPlats; i++) {
154                Plat p = menu[i];
155                String[] allergenes = p.getAllergenes();
156                String listeAllergenes = "";
157                for (int j = 0; j < allergenes.length; j++) {
158                    listeAllergenes += allergenes[j];
159                    if (j < allergenes.length - 1) listeAllergenes += SEPARATEUR_ALLERGENES;
160                }
161                out.write(p.getNom() + SEPARATEUR + p.getPrix() + SEPARATEUR
162                        + p.getDescription() + SEPARATEUR + listeAllergenes);
163                out.newLine();
164            }
165        }
166        System.out.println("Menu de " + nom + " ecrit dans " + fichier + " (" + nbPlats + " plats)");
167    }
```

Capture 5 — écriture : sauvegarderMenu (try-with-resources)

Pourquoi ces choix ?

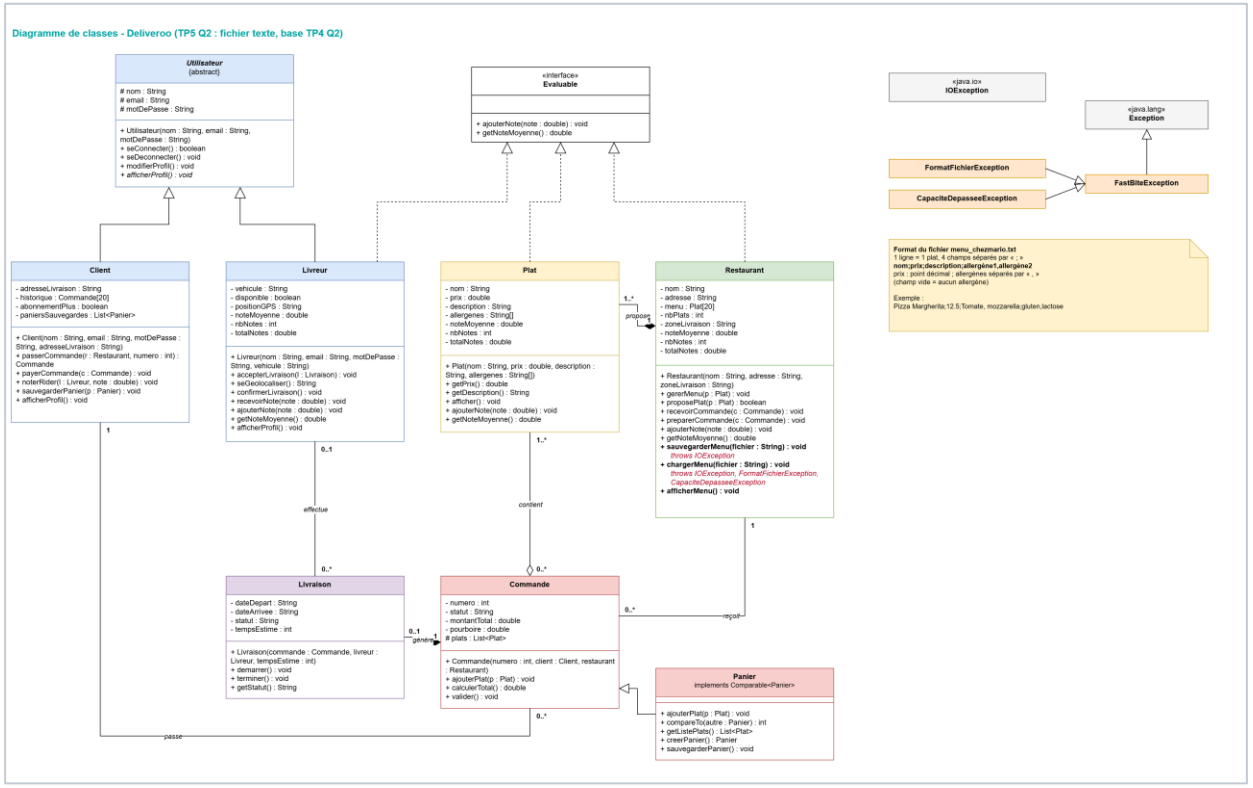
- try-with-resources : le fichier est fermé même en cas d'erreur.
- IOException propagée (throws) : l'appelant décide quoi faire.

• Restaurant.java

```
178    public void chargerMenu(String fichier)
179            throws IOException, FormatFichierException, CapaciteDepasseeException {
180        ArrayList<Plat> lus = new ArrayList<Plat>();
181
182        try (BufferedReader in = new BufferedReader(new FileReader(fichier))) {
183            String ligne;
184            int numeroLigne = 0;
185            while ((ligne = in.readLine()) != null) {
186                numeroLigne++;
187                if (ligne.trim().isEmpty()) continue;       // on ignore les lignes vides
188
189                String[] champs = ligne.split(SEPARATEUR, -1);   // -1 : garde le champ vide final
190                if (champs.length != 4) {
191                    throw new FormatFichierException(fichier, numeroLigne, ligne,
192                            "4 champs attendus, " + champs.length + " trouves");
193                }
194
195                double prix;
196                try {
197                    prix = Double.parseDouble(champs[1]);
198                } catch (NumberFormatException e) {
199                    // CHAINAGE : on garde l'erreur d'origine comme "cause"
200                    throw new FormatFichierException(fichier, numeroLigne, ligne,
201                            "prix non numerique", e);
202                }
203
204                String[] allergenes;
205                if (champs[3].isEmpty()) {
206                    allergenes = new String[0];
207                } else {
208                    allergenes = champs[3].split(SEPARATEUR_ALLERGENES);
209                }
210                lus.add(new Plat(champs[0], prix, champs[2], allergenes));
211            }
212        }
213
214        // Tout s'est bien passe : on peut modifier le menu
215        for (int i = 0; i < lus.size(); i++) {
216            gererMenu(lus.get(i));
217        }
218        System.out.println(lus.size() + " plat(s) lu(s) depuis " + fichier);
219    }
```

Capture 6 — lecture : chargerMenu (split, FormatFichierException)

Q2 — Résultat et UML mis à jour



Capture 7 — diagramme TP4 + sauvegarderMenu / chargerMenu / afficherMenu, IOException et FormatFichierException

menu_chezmario.txt - Bloc-notes

```
1  Pizza Margherita;12.5;Tomate, mozzarella, basilic;gluten,lactose
2  Pasta Carbonara;10.0;Lardons, oeuf, parmesan;gluten,oeuf,lactose
3  Eau minerale;2.0;Bouteille 50cl;
```

UTF-8 | 3 lignes | separateur : point-virgule

Capture 8 — le fichier produit (menu_chezmario.txt)

Windows PowerShell

```
PS ...\semaine 5\tp5_fastbite> java -cp out Main q2

================== Q2 - FICHIER TEXTE (.txt) ==================
Pizza Margherita ajoute au menu de Chez Mario
Pasta Carbonara ajoute au menu de Chez Mario
Eau minerale ajoute au menu de Chez Mario

>>> 1. ECRITURE du menu dans data/menu_chezmario.txt
Menu de Chez Mario ecrit dans data/menu_chezmario.txt (3 plats)
--- contenu brut de data/menu_chezmario.txt ---
1 | Pizza Margherita;12.5;Tomate, mozzarella, basilic;gluten,lactose
2 | Pasta Carbonara;10.0;Lardons, oeuf, parmesan;gluten,oeuf,lactose
3 | Eau minerale;2.0;Bouteille 50cl;
--- fin du fichier ---

>>> 2. LECTURE du fichier dans un nouveau restaurant, puis AFFICHAGE
Pizza Margherita ajoute au menu de Chez Mario (relu)
Pasta Carbonara ajoute au menu de Chez Mario (relu)
Eau minerale ajoute au menu de Chez Mario (relu)
3 plat(s) lu(s) depuis data/menu_chezmario.txt
Menu de Chez Mario (relu) (Paris 16) :
- Pizza Margherita : 12.5 EUR (Tomate, mozzarella, basilic) [allergenes : gluten, lactose]
- Pasta Carbonara : 10.0 EUR (Lardons, oeuf, parmesan) [allergenes : gluten, oeuf, lactose]
- Eau minerale : 2.0 EUR (Bouteille 50cl)

>>> 3. Fichier introuvable (IOException propagee puis attrapee)
Attrape (FileNotFoundException) : fichier introuvable -> data/inexistant.txt

>>> 4. Fichier mal forme : prix non numerique (FormatFichierException + cause)
--- contenu brut de data/menu_corrompu1.txt ---
1 | Pizza Margherita;12.5;Tomate, mozzarella;gluten,lactose
2 | Tiramisu;abc;Dessert maison;lactose
--- fin du fichier ---
Attrape : Fichier data/menu_corrompu1.txt, ligne 2 : prix non numerique [Tiramisu;abc;Dessert maison;lactose]
Cause d'origine : java.lang.NumberFormatException: For input string: "abc"

>>> 5. Fichier mal forme : nombre de champs incorrect
--- contenu brut de data/menu_corrompu2.txt ---
1 | Pizza Margherita;12.5;Tomate, mozzarella;gluten,lactose
2 | Tiramisu;5.5
--- fin du fichier ---
Attrape : Fichier data/menu_corrompu2.txt, ligne 2 : 4 champs attendus, 2 trouves [Tiramisu;5.5]
Le menu n'a pas ete modifie par les lectures ratees : 3 plats (comme avant)

PS ...\semaine 5\tp5_fastbite>
```

Capture 9 — écriture, relecture, affichage, puis 3 erreurs

Q3 — Sérialiser des restaurants

• Restaurant.java

```
25 public class Restaurant implements Evaluable, Serializable {
26     // Numero de version de la classe pour la serialisation
27     private static final long serialVersionUID = 1L;
```

• Restaurant.java

```
43     // TRANSIENT : compteur de session, volontairement NON sauvegarde par la serialisation.
44     // Apres une deserialisation il vaut 0 (valeur par defaut), c'est normal.
45     private transient int nbCommandesRecues;
```

• Plat.java

```
 9 public class Plat implements Evaluable, Serializable {
10     // Numero de version de la classe pour la serialisation
11     private static final long serialVersionUID = 1L;
```

Capture 10 — Serializable, serialVersionUID, champ transient

• Restaurant.java

```
241     public static void sauvegarderRestaurants(ArrayList<Restaurant> restaurants, String fichier)
242             throws IOException {
243         try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(fichier))) {
244             out.writeObject(restaurants);
245         }
246         System.out.println(restaurants.size() + " restaurant(s) serialise(s) dans " + fichier);
247     }
```

• Restaurant.java

```
256     @SuppressWarnings("unchecked")
257     public static ArrayList<Restaurant> chargerRestaurants(String fichier)
258             throws IOException, ClassNotFoundException {
259         try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(fichier))) {
260             ArrayList<Restaurant> restaurants = (ArrayList<Restaurant>) in.readObject();
261             System.out.println(restaurants.size() + " restaurant(s) deserialise(s) depuis " + fichier);
262             return restaurants;
263         }
264     }
```

Capture 11 — writeObject / readObject (cast + ClassNotFoundException)

Pourquoi ces choix ?

• Serializable sur Restaurant ET sur Plat : un objet n'est sérialisable que si tout ce qu'il contient l'est.

• serialVersionUID : fixe la version de la classe pour la relecture.

• transient : le compteur de session n'est pas enregistré (0 après relecture).

• Client, Livreur, Commande… non sérialisables : ce sont des sessions, pas des données.

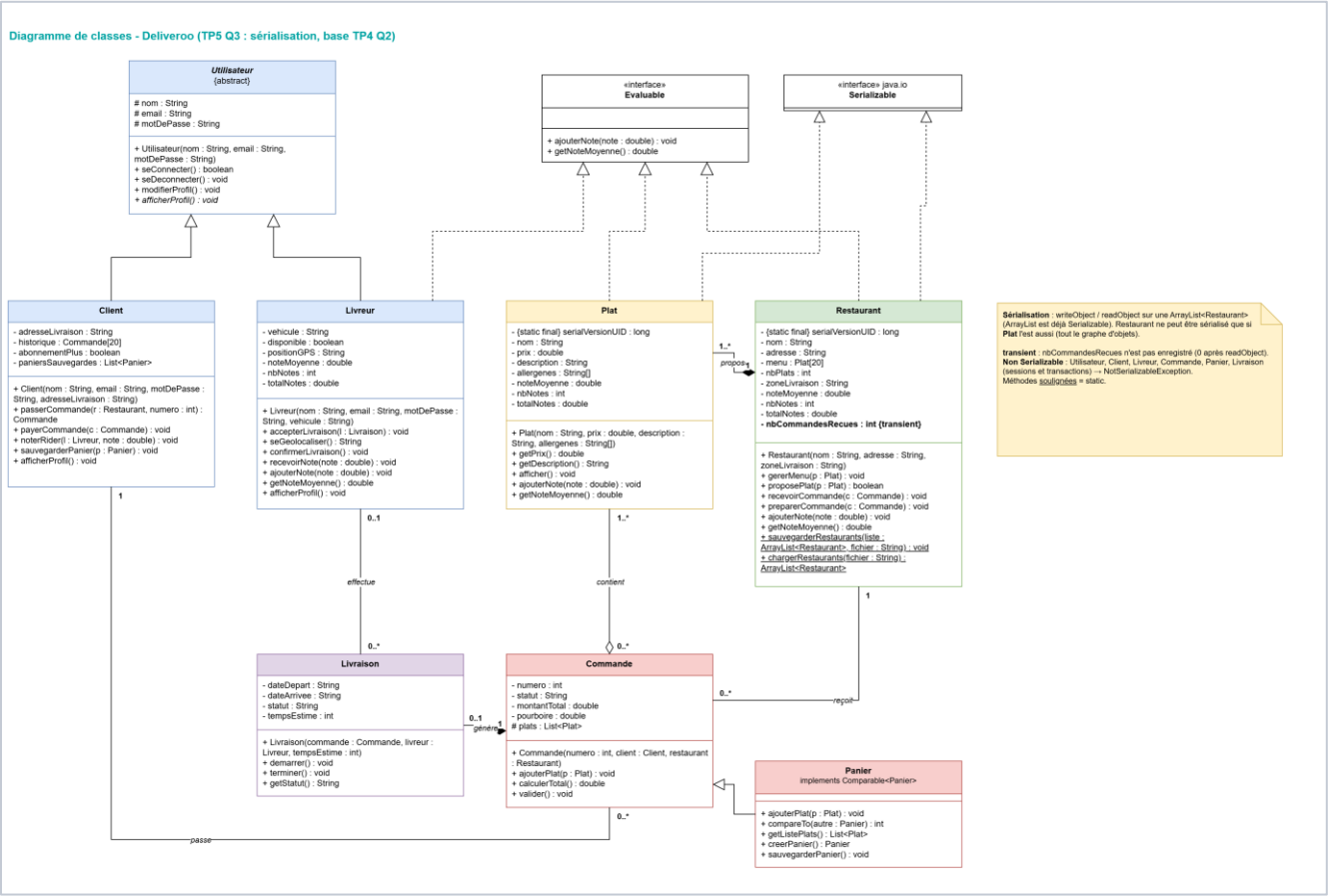
restaurants.ser - 872 octets (fichier binaire, 272 premiers octets affiches)

```
00000000  ac ed 00 05 73 72 00 13 6a 61 76 61 2e 75 74 69  ....sr..java.uti
00000010  6c 2e 41 72 72 61 79 4c 69 73 74 78 81 d2 1d 99  l.ArrayListx....
00000020  c7 61 9d 03 00 01 49 00 04 73 69 7a 65 78 70 00  .a....I..sizexp.
00000030  00 00 02 77 04 00 00 00 02 73 72 00 0a 52 65 73  ...w.....sr..Res
00000040  74 61 75 72 61 6e 74 00 00 00 00 00 00 00 01 02  taurant.........
00000050  00 08 49 00 07 6e 62 4e 6f 74 65 73 49 00 07 6e  ..I..nbNotesI..n
00000060  62 50 6c 61 74 73 44 00 0b 6e 6f 74 65 4d 6f 79  bPlatsD..noteMoy
00000070  65 6e 6e 65 44 00 0a 74 6f 74 61 6c 4e 6f 74 65  enneD..totalNote
00000080  73 4c 00 07 61 64 72 65 73 73 74 00 12 4c 6a     sL..adresset..Lj
00000090  61 76 61 2f 6c 61 6e 67 2f 53 74 72 69 6e 67 3b  ava/lang/String;
000000a0  5b 00 04 6d 65 6e 75 74 00 07 5b 4c 50 6c 61 74  [..menut..[LPlat
000000b0  3b 4c 00 03 6e 6f 6d 71 00 7e 00 03 4c 00 0d 7a  ;L..nomq.~..L..z
000000c0  6f 6e 65 4c 69 76 72 61 69 73 6f 6e 71 00 7e 00  oneLivraisonq.~.
000000d0  03 78 70 00 00 00 02 00 00 00 02 40 11 00 00 00  .xp........@....
000000e0  00 00 00 40 21 00 00 00 00 00 00 74 00 0f 31 32  ...@!......t..12
000000f0  20 72 75 65 20 64 65 20 50 61 72 69 73 75 72 00   rue de Parisur.
00000100  07 5b 4c 50 6c 61 74 3b bc d2 23 54 9f b9 8f 6a  .[LPlat;..#T...j
```

ac ed 00 05 = signature d'un flux Java serialise ; on y lit les noms de classes (ArrayList, Restaurant, Plat...) et les valeurs.

Capture 12 — le fichier .ser (binaire, signature ac ed 00 05)

Q3 — Résultat et UML mis à jour



Capture 13 — diagramme TP4 + interface Serializable (Restaurant, Plat), transient, static

```
Windows PowerShell

PS ...\semaine 5\tp5_fastbite> java -cp out Main q3

================== Q3 - SERIALISATION (.ser) ==================
Pizza Margherita ajoute au menu de Chez Mario
Pasta Carbonara ajoute au menu de Chez Mario
Chez Mario a recu la commande #9000
Sushi saumon ajoute au menu de Sushi Tokyo

>>> 1. Objets AVANT serialisation
 * Chez Mario [Paris 16] - note : 4.25/5 - commandes recues (transient) : 1
     - Pizza Margherita : 12.5 EUR (Tomate, mozzarella, basilic) [allergenes : gluten, lactose]
     - Pasta Carbonara : 10.0 EUR (Lardons, oeuf, parmesan) [allergenes : gluten, oeuf, lactose]
 * Sushi Tokyo [Paris 16] - note : pas encore note - commandes recues (transient) : 0
     - Sushi saumon : 9.0 EUR (Riz, saumon) [allergenes : poisson]

>>> 2. SERIALISATION dans data/restaurants.ser
2 restaurant(s) serialise(s) dans data/restaurants.ser
Taille du fichier : 872 octets (binaire)

>>> 3. DESERIALISATION et affichage des objets recuperes
2 restaurant(s) deserialise(s) depuis data/restaurants.ser
 * Chez Mario [Paris 16] - note : 4.25/5 - commandes recues (transient) : 0
     - Pizza Margherita : 12.5 EUR (Tomate, mozzarella, basilic) [allergenes : gluten, lactose]
     - Pasta Carbonara : 10.0 EUR (Lardons, oeuf, parmesan) [allergenes : gluten, oeuf, lactose]
 * Sushi Tokyo [Paris 16] - note : pas encore note - commandes recues (transient) : 0
     - Sushi saumon : 9.0 EUR (Riz, saumon) [allergenes : poisson]
Meme objet en memoire (==) ? false   -> c'est une COPIE
Meme nom ? true
Note du plat 'Pizza Margherita' : 5.0/5 (conservee)
Le champ transient nbCommandesRecues : 1 avant, 0 apres (non sauvegarde)

>>> 4. Classe NON Serializable (NotSerializableException)
Attrape : la classe Client n'implemente pas Serializable

>>> 5. Fichier .ser corrompu (StreamCorruptedException)
Attrape (StreamCorruptedException) : invalid stream header: 63656369

PS ...\semaine 5\tp5_fastbite>
```

Capture 14 — avant, sérialisation, désérialisation : copie identique, transient remis à 0

Bilan, sources et IA

Bilan

• Fait : 9 exceptions métier + hiérarchie, fichier texte (écrire, lire, afficher), sérialisation d'une liste de restaurants.

• Vérifié : TestsTP5 (tests du TP4 adaptés + nouveaux cas) : tous passent ; démonstration Main q1 / q2 / q3.

• Appris : checked / unchecked, throws et redéfinition, try-with-resources, chaînage de cause, transient, copie à la désérialisation.

• Difficultés : Panier et le throws hérité ; ne pas laisser un menu à moitié chargé.

• Limites : tableaux de 20 conservés (TP4) ; pas de « ; » dans une description ; pas de gestion de versions du .ser.

• Améliorations : listes dynamiques (ArrayList) à la place des tableaux ; format CSV/JSON standard.

Sources

• J.-P. Segado, TP5 « Exceptions – Fichier – Sérialisation » et cours 5-6 (ECE).

• Documentation Java SE (Oracle) : Exceptions, Serializable, ObjectOutputStream.

• Notre TP3 et TP4 (Question 4 complet), diagrammes draw.io de l'équipe.

IA utilisée

• Claude (Anthropic) : aide à l'écriture du code, des diagrammes et des slides. Code compilé et testé ; chaque choix est expliqué dans les notes de l'oral.

Exemples de prompts :

• « fait moi le tp 5, dis-moi ce que tu as fait et prends les captures d'écran »

• « à partir de TP4 question complet (fichier zip), fais les changements s'il y en a »